

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	Deepa preya H	Reg.No.:	192421047
Department:	Information technology	Date:	6/08/2026

Code of Conduct:

I Deepa preya H / 192421047 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce

bottlenecks.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Register Transfer Operations and ALU Operations

The experiment requires register transfers and arithmetic/logic operations such as **addition, subtraction, AND and OR**, demonstrating the role of the ALU.

Aim

To design and simulate a simple processor datapath that performs register transfer operations and basic arithmetic and logical operations using multiple registers and an ALU.

Objectives

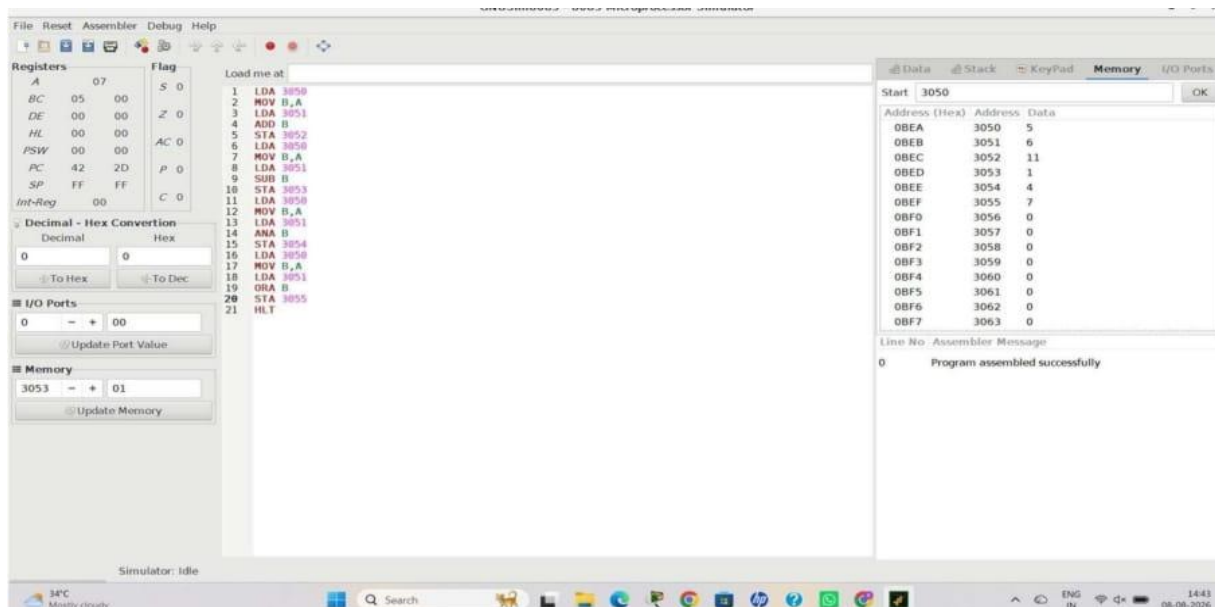
- To understand register-to-register data transfer.
- To perform arithmetic operations using the ALU.
- To perform logical AND and OR operations.
- To understand how registers and ALU work together.
- To observe the sequence of data movement during instruction execution.

Components Required

- Registers: R1, R2, R3, R4
- Arithmetic Logic Unit (ALU)
- Control Unit
- Internal data bus
- Multiplexer
- Clock
- Simulation software

Procedure

1. Initialize all registers.
2. Load values into R1 and R2.
3. Select the required ALU operation.
4. Transfer the ALU result to R3.
5. Perform register-to-register transfers.
6. Repeat for addition, subtraction, AND and OR.
7. Record the output.



Conclusion

The experiment demonstrates that the **ALU performs arithmetic and logical operations**, while registers store operands and results. Register-transfer operations allow data to move between different parts of the datapath.

Experiment 2: Single-Bus and Multi-Bus Organization

The experiment focuses on creating single-bus and multi-bus computer organizations, performing data transfers between registers, memory and I/O, and comparing their time and efficiency.

Aim

To design and simulate single-bus and multi-bus computer organizations and compare their data transfer performance and efficiency.

Objectives

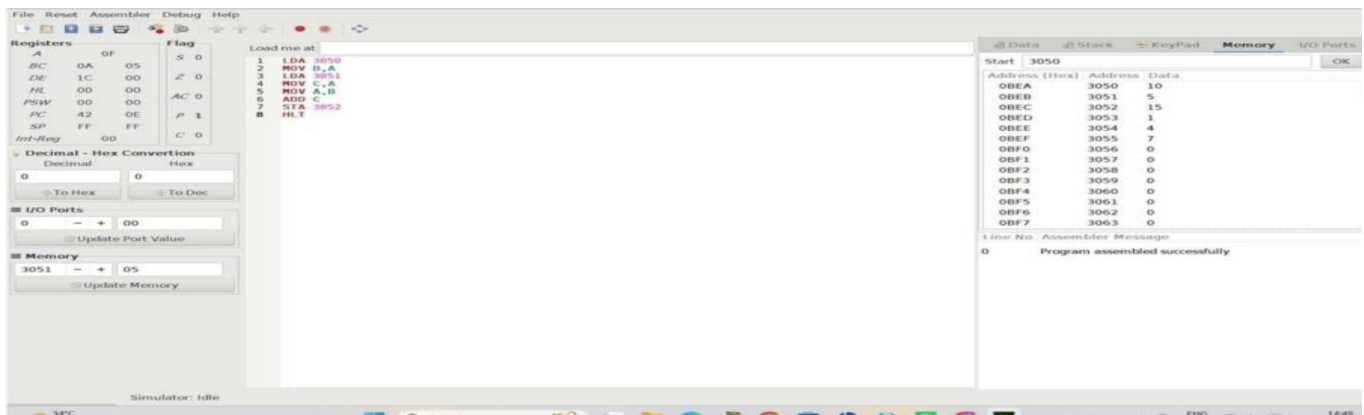
- To understand single-bus and multi-bus organization.
- To perform data transfer between registers, memory and I/O.
- To compare the execution time of both organizations.
- To understand how multiple buses improve parallel data transfer.

Components Required

- CPU/Processor
- Registers (R1, R2, R3, R4)
- ALU
- Memory Unit
- Input/Output Devices
- Bus
- Control Unit
- Clock
- Simulation Software

Procedure

1. Start the simulation software.
2. Create registers, ALU, memory and I/O units.
3. Connect the components using a single bus.
4. Initialize the registers with sample values.
5. Perform register-to-register data transfer.
6. Perform register-to-memory and memory-to-register transfers.
7. Perform I/O data transfers.
8. Record the execution time and clock cycles.
9. Create a multi-bus organization using two or more buses.
10. Repeat the same operations using the multi-bus system.
11. Record the execution time and compare both systems.



Result

The single-bus and multi-bus organizations were successfully simulated. The multi-bus organization provided **faster and more parallel data transfer** with reduced bottlenecks.

Experiment 3: Five-Stage Instruction Pipeline

The experiment requires simulation of a **5-stage instruction pipeline — Fetch, Decode, Execute, Memory and Write-back** — and comparison with a non-pipelined system using speedup and throughput.

Aim

To simulate a 5-stage instruction pipeline and compare its performance with a non-pipelined system in terms of **speedup and throughput**.

Objectives

- To understand the concept of instruction pipelining.
- To implement the five stages of a pipeline.
- To execute a set of instructions.
- To calculate speedup and throughput.
- To compare pipelined and non-pipelined execution.

Components Required

- CPU/Processor
- Program Counter (PC)
- Instruction Memory
- Registers
- ALU
- Data Memory
- Control Unit
- Pipeline Registers
- Clock
- Simulation Software

Procedure

1. Start the simulation software.
2. Create the five pipeline stages: IF, ID, EX, MEM and WB.
3. Initialize the registers and instruction memory.
4. Enter a set of instructions.

5. Fetch the first instruction from memory.
6. Decode the instruction and identify its operands.
7. Execute the operation using the ALU.
8. Perform the required memory operation.
9. Write the result back to the register.
10. Allow subsequent instructions to enter the pipeline while previous instructions move to the next stage.
11. Record the number of clock cycles required.
12. Execute the same instructions in a non-pipelined system.
13. Calculate speedup and throughput.
14. Compare the results.



Result

The five-stage instruction pipeline was successfully simulated. The pipelined system required **9 cycles** compared with **25 cycles** for the non-pipelined system for five instructions, giving an ideal speedup of approximately **2.78×**.

Experiment 4: Pipeline Hazards

This experiment requires creating a pipeline scenario involving **data, control and structural hazards**, and resolving them using **stalling, data forwarding and branch prediction**.

Aim

To simulate different types of pipeline hazards and implement techniques to resolve them and improve processor performance.

Objectives

- To understand data, control and structural hazards.
- To observe the effect of hazards on pipeline execution.
- To implement stalling and data forwarding.
- To understand branch prediction.
- To compare pipeline performance before and after hazard resolution.

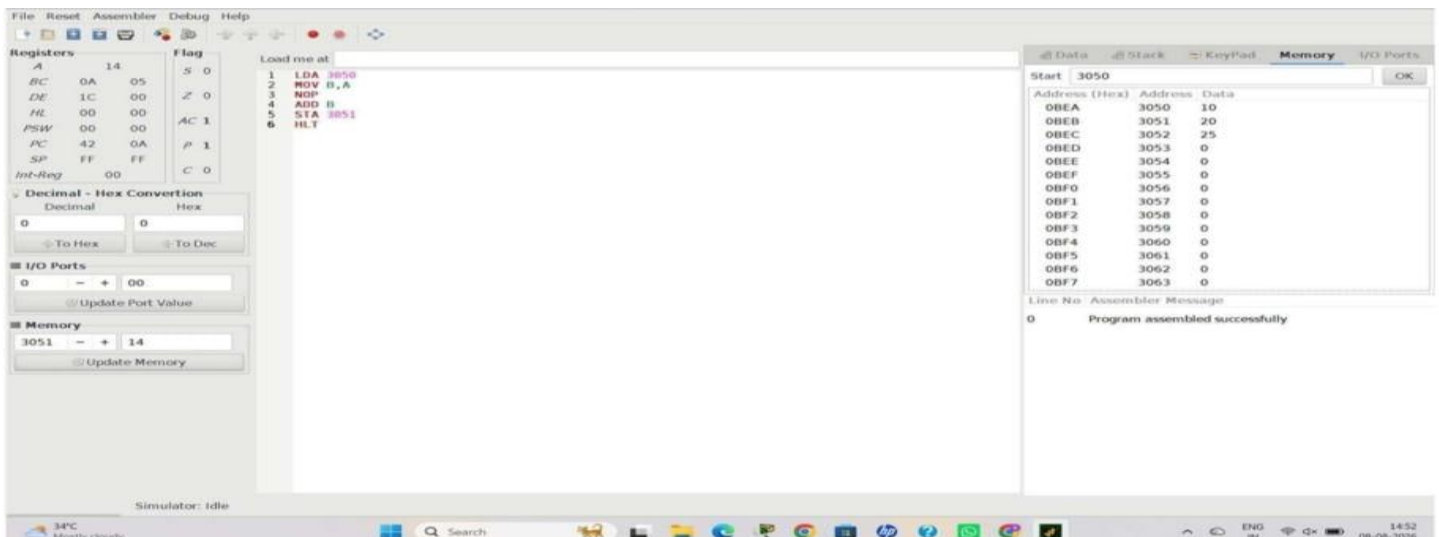
Components Required

- CPU/Processor
- Registers
- ALU
- Instruction Memory

- Data Memory
- Pipeline Registers
- Forwarding Unit
- Hazard Detection Unit
- Branch Prediction Unit
- Control Unit
- Clock
- Simulation Software

Procedure

1. Start the pipeline simulation software.
2. Create the five stages: **IF, ID, EX, MEM and WB**.
3. Load a set of instructions into instruction memory.
4. Create a data dependency between two instructions to produce a data hazard.
5. Observe the pipeline stall caused by the dependency.
6. Implement **data forwarding** and observe the improvement.
7. Add a branch instruction to create a control hazard.
8. Implement **branch prediction**.
9. Create a resource conflict to demonstrate a structural hazard.
10. Resolve the structural hazard using stalling or additional resources.
11. Record the number of stalls and clock cycles.
12. Compare pipeline execution before and after hazard resolution.



Result

The three types of pipeline hazards were successfully simulated. **Data forwarding, stalling and branch prediction** were implemented to reduce the negative effects of hazards on pipeline performance.

Experiment 5: Superscalar, Out-of-Order, Speculative Execution and SMT

This experiment focuses on analyzing **superscalar execution, out-of-order execution, speculative execution, hyper-threading and simultaneous multithreading (SMT)**, and comparing their effect on instruction throughput and resource utilization.

Aim

To analyze a processor supporting superscalar, out-of-order and speculative execution, along with hyper-threading and SMT, and evaluate their effect on processor performance.

Objectives

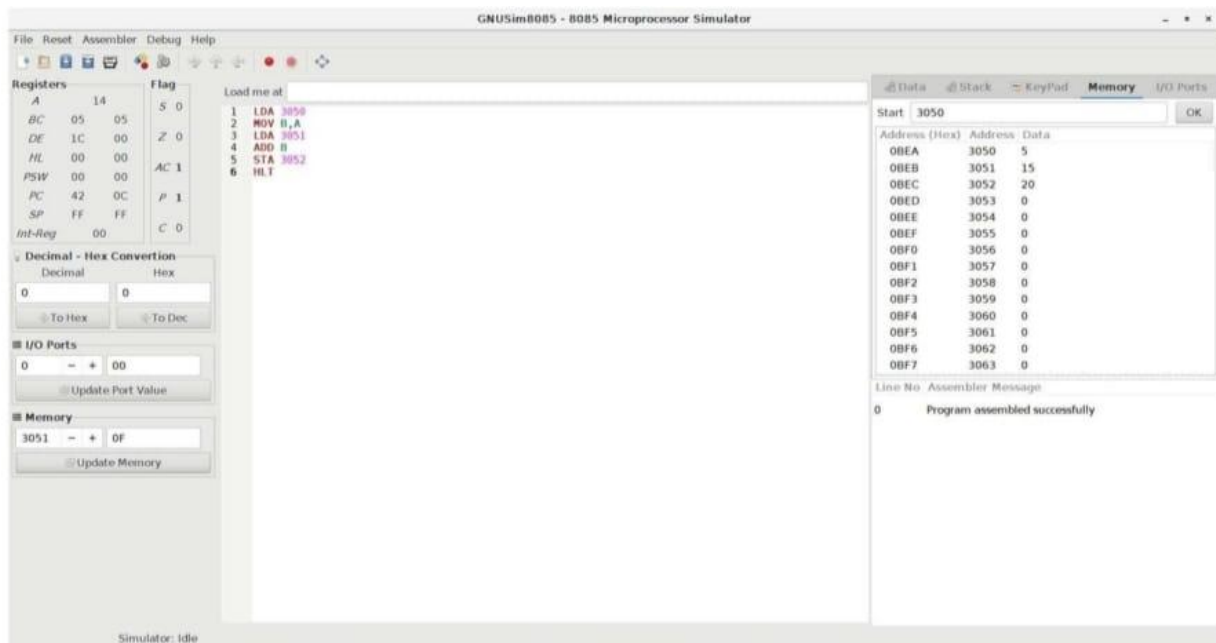
- To understand superscalar processor execution.
- To study out-of-order execution.
- To understand speculative execution.
- To study hyper-threading and SMT.
- To compare instruction throughput and resource utilization.

Components Required

- CPU/Processor
- Multiple ALUs / Execution Units
- Registers
- Instruction Memory
- Data Memory
- Instruction Decoder
- Control Unit
- Reorder Buffer
- Reservation Stations
- Branch Prediction Unit
- Multiple Threads
- Clock
- Simulation Software

Procedure

1. Start the processor simulation software.
2. Configure multiple execution units.
3. Enable superscalar execution.
4. Load a set of independent and dependent instructions.
5. Execute multiple instructions in parallel.
6. Enable out-of-order execution and observe the execution sequence.
7. Configure branch prediction for speculative execution.
8. Execute instructions speculatively and observe the result.
9. Configure two or more threads to demonstrate hyper-threading/SMT.
10. Execute instructions from multiple threads simultaneously.
11. Record the number of instructions executed per cycle.
12. Compare throughput and resource utilization for different execution techniques.



Result

The processor execution techniques were successfully analyzed. Superscalar, out-of-order, speculative execution and SMT improved the **instruction throughput and utilization of processor resources** compared with basic sequential execution.